

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.***Session 2026-2027**

Vision: Dream of where you want.	Mission: Means to achieve Vision
---	---

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	Preparation	P: Preparation	Pep-CL abbreviation pronounce as Pep-si-IL easy to recall
PEO2	Core Competence	E: Environment (Learning Environment)	
PEO3	Breadth	P: Professionalism	
PEO4	Professionalism	C: Core Competence	
PEO5	Learning Environment	L: Breadth (Learning in diverse areas)	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:

Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

“I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life.” to contribute to the development of cutting-edge technologies and Research.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Sheikh Wasimuddin
Name of Student

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.*

Session	2026-27 (ODD)	Course Name	PE-V : Lab : Blockchain Technology
Semester	7	Course Code	23IOT1748
Roll No	61	Name of Student	Sheikh Wasimuddin

Practical Number	06
Course Outcome	1. Demonstrate basics of distributed systems and cryptographic primitives used in blockchain. 2. Outline various research aspects for development of blockchain. 3. Make use of different security protocols to build secure blockchain applications. 4. Analyze smart contracts and Hyperledger Fabric used in blockchain. 5. Determine the performance of applications developed with Bitcoin and Ethereum blockchain.
Aim	Implement a Merkle Tree for Data Integrity Verification by Constructing the Tree, Generating Root Hashes, and Verifying Data Blocks.
Theory (100 words)	A Merkle Tree is a cryptographic tree structure that is used to securely organize and verify data. It is built using a cryptographic hash function such as SHA-256, where each data block is converted into a fixed-length hash value. These hash values are placed at the bottom level of the tree as leaf nodes. The hashes of two neighboring nodes are then combined and passed through the hash function again to produce a parent node. This process is repeated for every level until a single hash remains at the top, which is called the Merkle Root. The Merkle Root serves as a digital fingerprint of all the data stored in the tree. The main purpose of a Merkle Tree is to provide data integrity and efficient verification. If even a single character or value in a data block is changed, its hash value also changes. This change propagates through the parent nodes and ultimately produces a different Merkle Root. By comparing the current Merkle Root with the previously stored root, it is possible to determine whether the data has been altered. This provides a reliable method for detecting unauthorized modifications. Merkle Trees also reduce the amount of data required for verification. Instead of comparing the complete dataset, a user can verify a particular block by using its hash and the hashes of the neighboring nodes along the path to the root. This is known as a Merkle Proof. Because of this property, Merkle Trees are especially useful when working with large datasets and distributed systems where storing or transferring the complete data for

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.*

	verification would be inefficient. Merkle Trees are an important component of blockchain technology. In many blockchain systems, transactions within a block are organized into a Merkle Tree, and the resulting Merkle Root is stored in the block header. This allows nodes to efficiently verify whether a particular transaction belongs to a block without processing every transaction. Merkle Trees are also used in distributed databases, peer-to-peer networks, cloud storage, version-control systems, and other applications where fast and secure data verification is required.
Procedure and Execution (100 Words)	1. Open Remix IDE and create a new Solidity file named StudentResult.sol. 2. Specify the required Solidity compiler version using the pragma statement. 3. Define a smart contract named StudentResult. 4. Declare variables to store the student name, student ID, and marks in three subjects. 5. Create a function to enter and store the student's details and marks. 6. Create a function to calculate the total marks by adding the marks obtained in all three subjects. 7. Create a function to calculate the average marks by dividing the total marks by 3. 8. Use an if-else conditional statement to determine the student's result. If the average is 40 or above, the result is Pass; otherwise, it is Fail. 9. Create a function to display the student name, student ID, subject marks, total marks, average marks, and final result. 10. Open the Solidity Compiler tab in Remix IDE and compile the StudentResult.sol contract. 11. Go to Deploy & Run Transactions and select the StudentResult contract. 12. Deploy the smart contract using the available Remix environment, such as Remix VM. 13. Enter the student's information and marks by calling the appropriate input function. 14. Call the total and average calculation functions and verify the calculated values. 15. Call the result function and verify whether the student is declared Pass or Fail based on the average marks. 16. Verify that the displayed student information and calculated result are correct.



Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

Stepwise Screenshots:

```
#sheikh wasimuddin 61 CSEIOT lab 6 BT
import hashlib

def hash_data(data):
    return hashlib.sha256(data.encode()).hexdigest()

def build_merkle_tree(data_blocks):
    tree = [hash_data(data) for data in data_blocks]

    while len(tree) > 1:
        if len(tree) % 2 != 0:
            tree.append(tree[-1])

        new_tree = []

        for i in range(0, len(tree), 2):
            combined = tree[i] + tree[i + 1]
            new_tree.append(hash_data(combined))

        tree = new_tree

    return tree[0]

# Data blocks
data_blocks = [
    "Wasimuddin",
    "Tanmay",
    "Akare",
    "Parth"
]

# Generate Merkle Root
root_hash = build_merkle_tree(data_blocks)

print("Merkle Root:", root_hash)

# Verify data
data = "Wasimuddin"

if data in data_blocks:
    print("Data is valid")
else:
    print("Data is not valid")
```



```
def hash_data(data):
    return hashlib.sha256(data.encode()).hexdigest()

def build_merkle_tree(data_blocks):
    tree = [hash_data(data) for data in data_blocks]

    while len(tree) > 1:
        if len(tree) % 2 != 0:
            tree.append(tree[-1])

        new_tree = []

        for i in range(0, len(tree), 2):
            combined = tree[i] + tree[i + 1]
            new_tree.append(hash_data(combined))

        tree = new_tree

    return tree[0]

# Data blocks
data_blocks = [
    "Wasimuddin",
    "Tanmay",
    "Akare",
    "Parth"
]

# Generate Merkle Root
root_hash = build_merkle_tree(data_blocks)

print("Merkle Root:", root_hash)

# Verify data
data = "Wasimuddin"

if data in data_blocks:
    print("Data is valid")
else:
    print("Data is not valid")
```

... Merkle Root: 7413897aa477b78e3102e3c2bb305cb5b38d4a2290261a565a58670149398d98
Data is valid

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.*

	<pre>def hash_data(data): return hashlib.sha256(data.encode()).hexdigest() def build_merkle_tree(data_blocks): tree = [hash_data(data) for data in data_blocks] while len(tree) > 1: if len(tree) % 2 != 0: tree.append(tree[-1]) new_tree = [] for i in range(0, len(tree), 2): combined = tree[i] + tree[i + 1] new_tree.append(hash_data(combined)) tree = new_tree return tree[0] # Data blocks data_blocks = ["Wasimuddin", "Tanmay", "Akare", "Parth"] # Generate Merkle Root root_hash = build_merkle_tree(data_blocks) print("Merkle Root:", root_hash) # Verify data data = "Vedant" if data in data_blocks: print("Data is valid") else: print("Data is not valid") ... Merkle Root: 7413897aa477b78e3102e3c2bb305cb5b38d4a2290261a565a58670149398d98 Data is not valid</pre>
Code	<pre>#sheikh wasimuddin 61 CSEIOT lab 6 BT import hashlib def hash_data(data): return hashlib.sha256(data.encode()).hexdigest() def build_merkle_tree(data_blocks): tree = [hash_data(data) for data in data_blocks] while len(tree) > 1: if len(tree) % 2 != 0: tree.append(tree[-1]) new_tree = [] for i in range(0, len(tree), 2): combined = tree[i] + tree[i + 1] new_tree.append(hash_data(combined))</pre>

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.*

	<pre>tree = new_tree return tree[0] # Data blocks data_blocks = ["Wasimuddin", "Tanmay", "Akare", "Parth"] # Generate Merkle Root root_hash = build_merkle_tree(data_blocks) print("Merkle Root:", root_hash) # Verify data data = "Vedant" if data in data_blocks: print("Data is valid") else: print("Data is not valid")</pre>
Conclusion	The Merkle Tree was successfully implemented using the SHA-256 cryptographic hashing algorithm. The given data blocks were converted into hash values and combined recursively to generate the Merkle Root. The root hash provides a unique and compact representation of the complete dataset and can be used to detect any modification in the data. The verification process also demonstrated how a particular data block can be checked efficiently against the original dataset. Therefore, the practical successfully demonstrated the use of Merkle Trees for data integrity, secure verification, and tamper detection, which are important applications in blockchain and distributed systems
Date	08/09/2026